Social Network Analysis
Ch1: Introduction

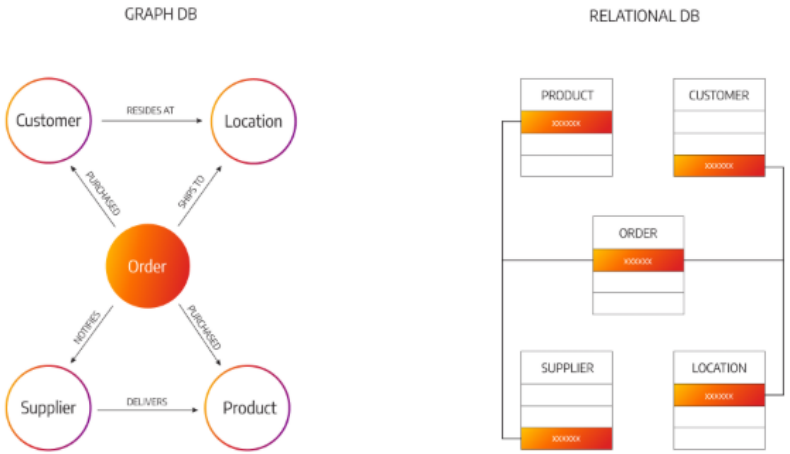
Ekarat Rattagan

https://ekaratnida.github.io/

Why SNA

In a standard relational database (like a table of SQL rows and columns), data points are stored in isolation. If you want to know how things connect, you have to link tables together using a JOIN command. For simple questions, that works. But the moment you need to look deep into relationships, tables become incredibly slow, complicated, and rigid.

Graph analysis treats the relationships *between* data points as just as important as the data points themselves.



Use case

- **Financial fraud detection** (finding networks of fake accounts),
- **Customer 360** (mapping a customer’s journey across multiple devices and product subscriptions),
- **Supply chain traceability** (tracking parts from raw materials down to the finished product line).

Use case

Imagine you work at a bank and suspect a stolen credit card is being used. You ask your database:

> *"Find all accounts that used the same phone number as the stolen card, then see what other devices those accounts logged into, and then list all other names associated with those devices."*

- **The SQL Way:** To answer this, your database has to perform 4 or 5 consecutive table JOIN operations across millions of rows. This results in massive, messy queries that can take minutes to process or completely crash your system under heavy load.
- **The Graph Way:** Because a graph stores data as nodes (entities) and edges (physical links), a graph engine doesn't need to search through rows. It simply starts at the "stolen card" node and walks down the existing paths (called "traversal"). It answers a 5-hop question in milliseconds.

Graph Theory (Introduction)

The mathematical study of networks.

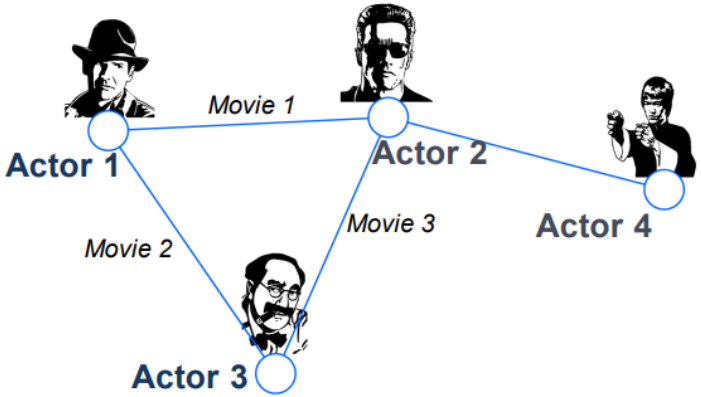
1. A **Node** (**Vertex**) represents an individual entity or object in your network.

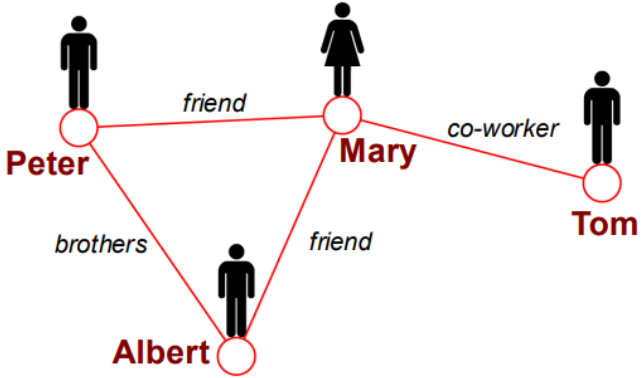
- *Examples:* A person, a web page, a city, a molecule, or a bank account.
- *Mathematical Notation:* The set of all vertices in a graph is written as V.

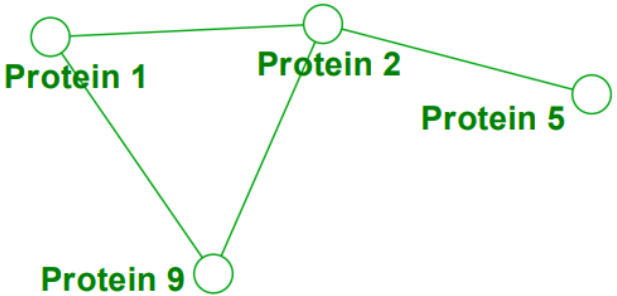
2. An **Edge** represents a relationship, connection, or interaction between two nodes.

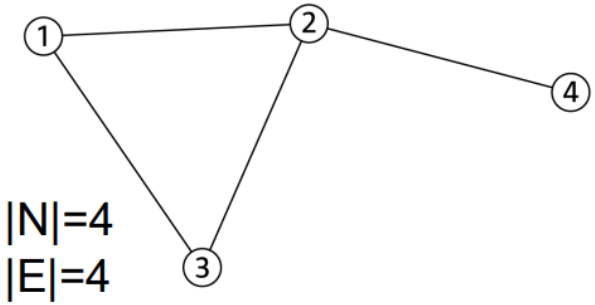
- *Examples:* A friendship, a hyperlink, a physical road, a chemical bond, or a financial transaction.
- *Mathematical Notation:* The set of all edges is written as E.

A graph G is mathematically defined as an ordered pair of these sets $G = (V, E)$



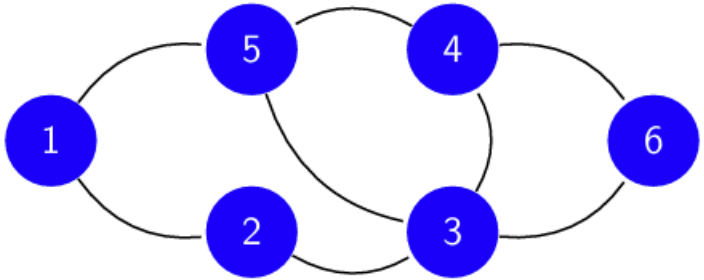






https://web.stanford.edu/class/cs224w/slides/01-intro.pdf

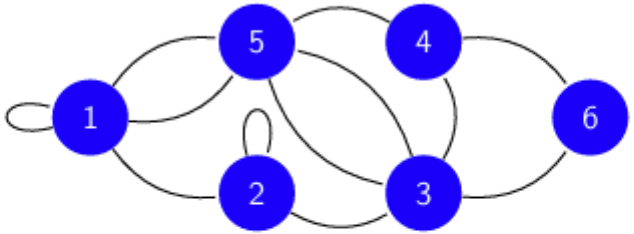
Graph



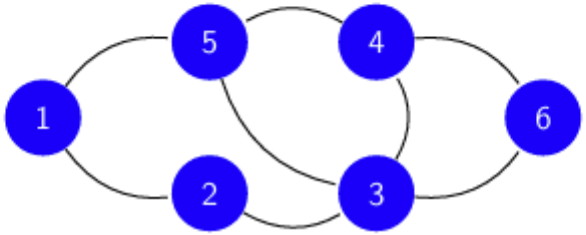
- Graph $G(V, E)$ $\Rightarrow$ A set V of vertices or nodes
  $\Rightarrow$ Connected by a set E of edges or links
  $\Rightarrow$ Elements of E are unordered pairs (u, v), $u, v \in V$
- In figure $\Rightarrow$ Vertices are $V = \{1, 2, 3, 4, 5, 6\}$
  $\Rightarrow$ Edges $E = \{(1, 2), (1, 5), (2, 3), (3, 4), \ldots$
  $(3, 5), (3, 6), (4, 5), (4, 6)\}$

Simple and Multi-Graphs

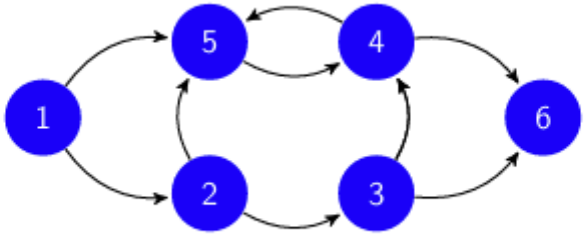
- In general, graphs may have self-loops and multi-edges
  - $\Rightarrow$ A graph with either is called a multi-graph



- Mostly work with simple graphs, with no self-loops or multi-edges



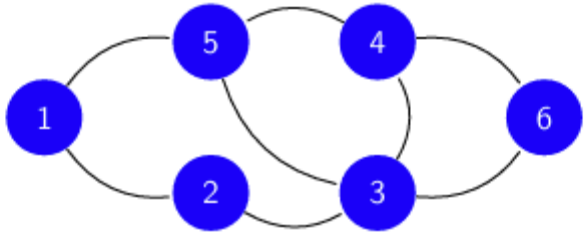
Directed Graphs



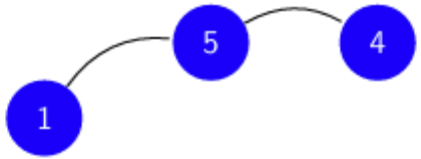
- In directed graphs, elements of E are ordered pairs (u, v), $u, v \in V$
  - $\Rightarrow$ Means (u, v) distinct from (v, u)
- Directed graphs often called digraphs
  - $\Rightarrow$ By convention (u, v) points to v
  - $\Rightarrow$ If both $\{(u, v), (v, u)\} \subseteq E$, the edges are said to be mutual
- Ex: who-calls-whom phone networks, Twitter follower networks

Subgraphs

- Consider a given graph $G(V, E)$



- **Def:** Graph $G'(V', E')$ is an induced subgraph of G if $V' \subseteq V$ and $E' \subseteq E$ is the collection of edges in G among that subset of vertices
- Ex: Graph induced by $V' = \{1, 4, 5\}$



Weighted graphs

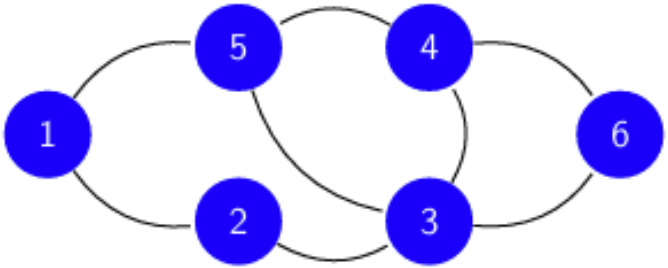
- Oftentimes one labels edges with numerical values
  - $\Rightarrow$ Such graphs are called weighted graphs
- Typical network representations:

Network	**Graph representation**
WWW	Directed multi-graph (with loops), unweighted
Facebook friendships	Undirected, unweighted
Citation network	Directed, unweighted, acyclic
Collaboration network	Undirected, unweighted
Mobile phone calls	Directed, weighted
Protein interaction	Undirected multi-graph (with loops), unweighted
⋮	⋮

- Note that multi-edges are often encoded as edge weights (counts)

Adjacency

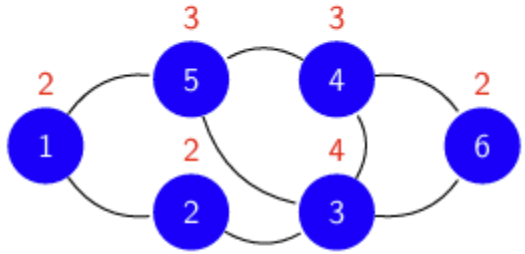
- Useful to develop a language to discuss the connectivity of a graph
- A simple and local notion is that of adjacency
  - $\Rightarrow$ Vertices $u, v \in V$ are said adjacent if joined by an edge in E
  - $\Rightarrow$ Edges $e_1, e_2 \in E$ are adjacent if they share an endpoint in V



- In figure $\Rightarrow$ Vertices 1 and 5 are adjacent; 2 and 4 are not
  $\Rightarrow$ Edge $(1,2)$ is adjacent to $(1,5)$, but not to $(4,6)$

Degree

- An edge (u, v) is incident with the vertices u and v
- **Def:** The degree d_v of vertex v is its number of incident edges



- In figure $\Rightarrow$ Vertex degrees shown in red, e.g., $d_1 = 2$ and $d_5 = 3$
- High-degree vertices likely influential, central, prominent.
- The neighborhood $\mathcal{N}_i$ of a node i is the set of all its adjacent nodes
  $\Rightarrow \mathcal{N}_5 = \{1, 3, 4\}$ $\Rightarrow$ In general, $|\mathcal{N}_i| = d_i$

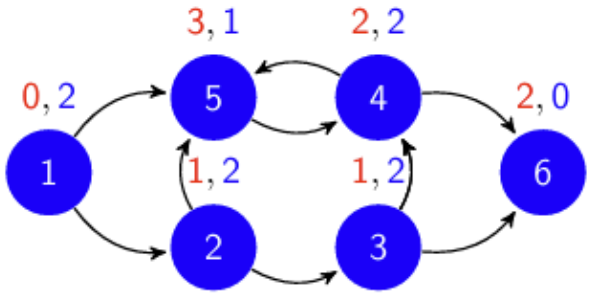
Properties and observations about degrees

- Degree values range from 0 to $|V| - 1$
- The sum of the degree sequence is twice the size of the graph

$$\sum_{v=1}^{|V|} d_v = 2|E|$$

$\Rightarrow$ The number of vertices with odd degree is even

- In digraphs, we have vertex in-degree d_v^{in} and out-degree d_v^{out}



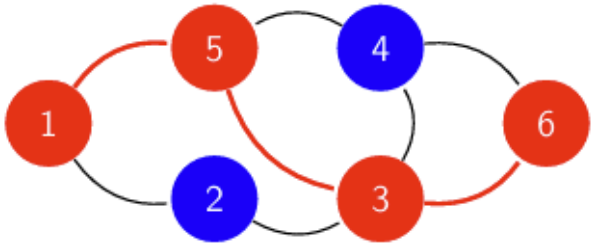
- In figure $\Rightarrow$ Vertex in-degrees shown in red, out-degrees in blue
  $\Rightarrow$ For example, $d_1^{in} = 0, d_1^{out} = 2$ and $d_5^{in} = 3, d_5^{out} = 1$

Movement in a graph

- A path of length l from v_0 to v_l is a consecutive sequence of **distinct** vertices

$$\{v_0, v_1, \ldots, v_{l-1}, v_l\}, \text{ where } v_i \text{ and } v_{i+1} \text{ are adjacent}$$

- A Walk: vertices do not have to be distinct.



- A closed walk ($v_0 = v_l$) is called a circuit

  $\Rightarrow$ A closed path is a cycle

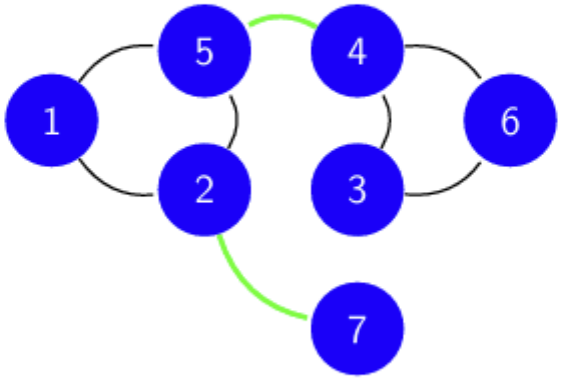
- All these notions generalize naturally to directed graphs

Distance in a graph

- Length of a path $\Rightarrow$ is the sum of the weights of traversed edges
- The distance between two nodes i and j is the length of the shortest path linking i and j
  - $\Rightarrow$ In the absence of such a path, the distance is ∞
  - $\Rightarrow$ The diameter of the graph is the value of the largest distance
- There exist efficient algorithms to compute distances in graphs
  - $\Rightarrow$ Dijkstra, Floyd-Warshall, Johnson, ...

Connectivity

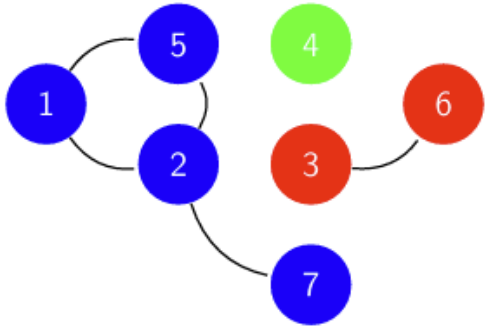
- Vertex v is reachable from u if there exists a $u - v$ path
- **Def:** Graph is connected if every vertex is reachable from every other



- If bridge edges are removed, the graph becomes disconnected

Connected components

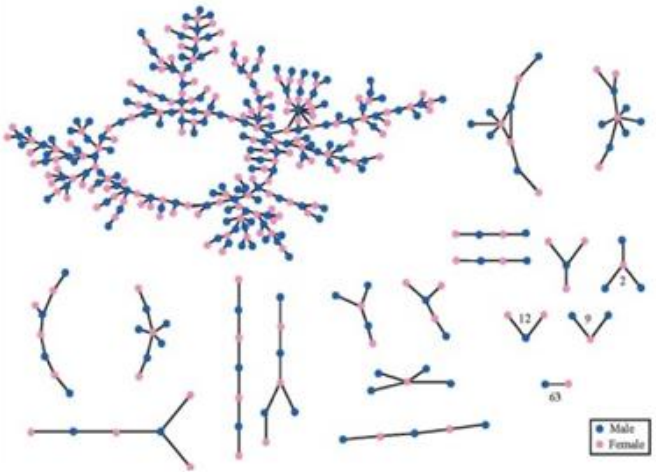
- **Def:** A component is a maximally connected subgraph
  - $\Rightarrow$ Maximal means adding a vertex will ruin connectivity



- In figure $\Rightarrow$ Components are $\{1,2,5,7\}$, $\{3,6\}$ and $\{4\}$
  - $\Rightarrow$ Subgraph $\{3,4,6\}$ not connected, $\{1,2,5\}$ not maximal
- Disconnected graphs have 2 or more components
  - $\Rightarrow$ Largest component often called giant component

Giant connected components

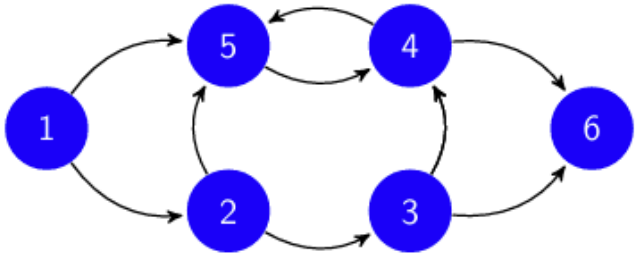
- Large real-world networks typically exhibit one giant component
- Ex: romantic relationships in a US high school [Bearman et al'04]



- Q: Why do we expect to find a single giant component?
- A: It only takes one edge to merge two giant components

Connectivity of directed graphs

- Connectivity is more subtle with directed graphs. Two notions
- Digraph is strongly connected if for every pair $u, v \in V$, u is reachable from v (via a directed path) and vice versa
- Digraph is weakly connected if connected after disregarding edge directions, i.e., the underlying undirected graph is connected

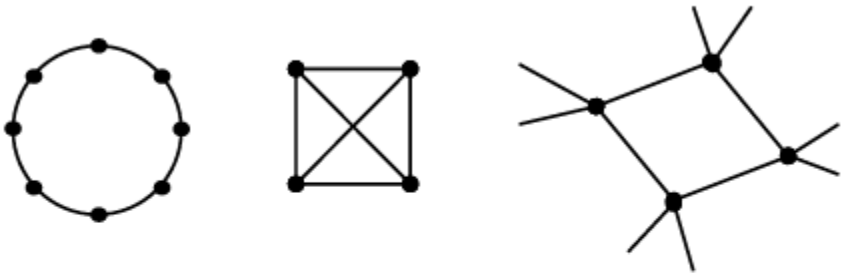


- Above graph is weakly connected but not strongly connected

  $\Rightarrow$ Strong connectivity implies weak connectivity

Regular graphs

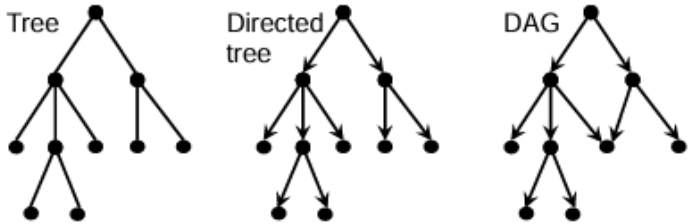
- A *d*-regular graph has vertices with equal degree *d*



- Naturally, the complete graph K_n is $(n-1)$-regular
  - $\Rightarrow$ Cycles are 2-regular (sub) graphs
- Regular graphs arise frequently in e.g.,
  - Physics and chemistry in the study of crystal structures
  - Geo-spatial settings as pixel adjacency models in image processing
  - Opinion formation, information cycles as regular subgraphs

Trees and directed acyclic graphs

- A tree is a connected acyclic graph
  - $\Rightarrow$ A collection of trees is denominated a forest
- Ex: river network, information cascades in Twitter, citation network



- A directed tree is a digraph whose underlying undirected graph is a tree
  - $\Rightarrow$ Rooted if paths from one vertex to all others
- Vertex terminology: parent, children, ancestor, descendant, leaf
- Underlying graph of a directed acyclic graph (DAG) need not be a tree

Matrix representation

Adjacency matrix

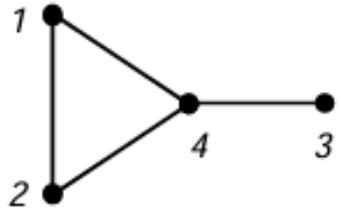
- Algebraic graph theory deals with matrix representations of graphs
  - $\Rightarrow$ Leverage algebra to 'visualize' graphs as if being plotted
- Q: How can we capture the connectivity of $G(V, E)$ in a matrix?
- A: Binary, symmetric adjacency matrix $\mathbf{A} \in \{0,1\}^{|V| \times |V|}$, with entries

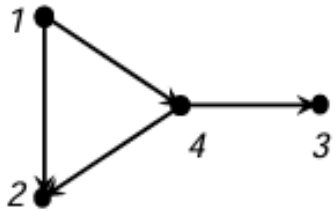
$$A_{ij} = \begin{cases} 1, & \text{if } (i,j) \in E \\ 0, & \text{otherwise} \end{cases} .$$

  - $\Rightarrow$ Note that vertices are indexed with integers $1, \ldots, |V|$
- In words, $\mathbf{A}$ is one for those entries whose row-column indices denote vertices in V joined by an edge in E, and is zero otherwise

Adjacency matrix examples

- Examples for undirected graphs and digraphs





$$\mathbf{A}_u = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}, \quad \mathbf{A}_d = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

- If the graph is weighted, store the (i, j) weight instead of 1

Adjacency matrix properties

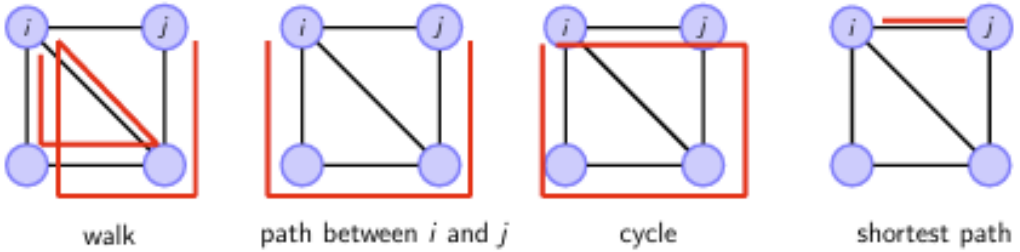
- Adjacency matrix useful to store graph structure.
  $\Rightarrow$ Also, operations on $\mathbf{A}$ yield useful information about G
- Degrees: Row-wise sums give vertex degrees, i.e., $\sum_{j=1}^{|V|} A_{ij} = d_i$
- For digraphs $\mathbf{A}$ is not symmetric and row-, colum-wise sums differ

$$\sum_{j=1}^{|V|} A_{ij} = d_i^{out}, \qquad \sum_{i=1}^{|V|} A_{ij} = d_j^{in}$$

- Spectrum: G is d-regular if and only if $\mathbf{1}$ is an eigenvector of $\mathbf{A}$, i.e.,

$$\mathbf{A1} = d\mathbf{1}$$

Walks, Paths, and Cycles



- Walks: Let $\mathbf{A}^r$ denote the r-th power of $\mathbf{A}$, with entries A^r_{ij}
- $[A^2]_{ij} := \sum_{k=1}^{n} A_{ik} A_{kj}$
- Corollary: $\mathrm{tr}(\mathbf{A}^2)/2 = |E|$ and $\mathrm{tr}(\mathbf{A}^3)/6 = \#\triangle$ in G

  $\Rightarrow$ You will prove this in your homework

Incidence matrix

- A graph can be also represented by its $|V| \times |E|$ incidence matrix $\mathbf{B}$
  $\Rightarrow$ $\mathbf{B}$ is in general not a square matrix, unless $|V| = |E|$
- For undirected graphs, the entries of $\mathbf{B}$ are

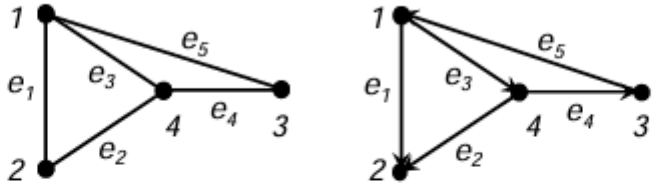
$$B_{ij} = \begin{cases} 1, & \text{if vertex } i \text{ incident to edge } j \\ 0, & \text{otherwise} \end{cases} .$$

- For digraphs we also encode the direction of the edge, namely

$$B_{ij} = \begin{cases} 1, & \text{if edge } j \text{ is } (k, i) \\ -1, & \text{if edge } j \text{ is } (i, k) \\ 0, & \text{otherwise} \end{cases} .$$

Incidence matrix examples

- Examples for undirected graphs and digraphs



$$\mathbf{B}_u = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{pmatrix}, \quad \mathbf{B}_d = \begin{pmatrix} -1 & 0 & -1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & -1 & 1 & -1 & 0 \end{pmatrix}$$

- If the graph is weighted, modify nonzero entries accordingly

Exercise

- https://colab.research.google.com/drive/1cblxr9EGPlr00M2yxOO36cbTBC3yid5z?usp=sharing